

American Sign Language Hand-Gesture Classification across 37 Classes: A Theoretical and Empirical Comparison of Scratch CNNs and Transfer Learning

Sinan Büyük, Michael Eith, and Jordi Xu Wang



I. INTRODUCTION

Sign languages are the main language of communication for a large part of the deaf and hard-of-hearing community, with the World Health Organization estimating that more than 5% of the world's population lives with disabling hearing loss [1]. American Sign Language (ASL) has some 500,000 users, according to estimates [2]. Automatic Sign Language Recognition (SLR) is an enabling technology for assistive communication devices, human-computer interaction and educational tools that convert signs into text or speech. A robust, low-latency gesture classifier is a building block to accessible interfaces running on commodity hardware without specialized sensors.

This project targets the *static, vision-based* sub-problem of fingerspelling recognition: classifying a single RGB frame of a hand into one of **37 classes**: the ten digits 1-10, the twenty-six letters *A-Z*, and a *blank* class denoting the absence of a sign. Rather than proposing a new architecture, the goal is to build a clean, reproducible pipeline grounded in the theory of convolutional networks, and to answer two practical questions empirically: (i) *how far can a convolutional network trained from scratch go on this data, and which design choices matter most?*, and (ii) *how much does ImageNet transfer learning help, and what does the resulting accuracy cost in model size?* We unify five heterogeneous public datasets, control for near-duplicate frames, and benchmark ten models under a common protocol, closing with a deployment-oriented analysis of the accuracy/size trade-off, a live real-time demonstration, and an honest discussion of the limitations introduced by the data. Throughout, we give the mathematical formulation of each component so that the design choices can be justified from theory rather than asserted.

Concretely, the contributions of this work are: (i) a unified, leakage-controlled 37-class ASL image benchmark assembled from five heterogeneous public sources; (ii) a controlled regularisation ablation that isolates the individual effect of depth, Dropout, Batch Normalisation and augmentation on identical scratch architectures; (iii) a like-for-like comparison of four ImageNet backbones fine-tuned under one protocol; and (iv) a deployment-oriented analysis that weighs accuracy against model size and demonstrates real-time feasibility with a live webcam recognition system built around the most efficient backbone, together with a theory-grounded interpretation of every observed effect.

II. STATE OF THE ART

Early vision-based SLR systems relied on hand-crafted descriptors, used appearance or depth cues provided to classical classifiers such as random forests or Support Vector Machines, and were prone to background, lighting and signer variation [5]. The field has moved almost entirely to deep learning since then. Recent surveys list convolutional, recurrent and graph-based approaches and generally conclude that learned features are better than engineered ones [3], [4].

Modern image classifiers are descendants of the convolutional architecture of LeNet [6] and the deep models that won

the ImageNet challenge [14], [15], starting with AlexNet [7] and the very deep but uniform VGG networks [8]. Residual networks (ResNet) [9] remain a strong default backbone, as skip connections enable reliable training of much deeper models. Training these networks involves two main components: Batch Normalisation [12], which normalises intermediate activations to stabilise and speed up optimisation, and Dropout [13], which mitigates overfitting by randomly deactivating units. We typically use Adam [16] for optimisation with cosine-annealed learning rate [17], and mixed-precision training [18] to reduce memory and wall-clock cost.

Labelled data is often limited, so transfer learning, initialising from ImageNet weights and fine-tuning, has become standard practice, and often outperforms training from scratch on medium-sized datasets [15]. For deployment, efficiency-oriented families such as MobileNetV3 [10] and EfficientNet [11] reach similar accuracy with significantly fewer parameters/training time. Another orthogonal line of work avoids raw-pixel classification by first extracting hand landmarks with real-time trackers such as MediaPipe Hands [20], and then classifying the resulting key-points. Our work is squarely in the pixel-based transfer learning regime: we are not suggesting a novel model, but instead offer a controlled comparison of scratch vs. transfer learning, and a regularization ablation on a unified multi-source ASL benchmark, evaluated using a deployment-oriented metric besides accuracy, and validated with a real-time webcam demonstrator. All models are implemented within PyTorch [19].

Within ASL fingerspelling specifically, two methodological camps coexist. Pixel-based classifiers operate directly on the hand image and learn discriminative features end-to-end. They are simple to deploy but must learn invariance to background and lighting from data. Landmark-based pipelines instead run a hand-pose estimator such as MediaPipe Hands [20] and classify the resulting key-points, gaining appearance invariance at the cost of a dependency on the upstream tracker and a loss of fine texture cues, for example the exact thumb contact that separates *M*, *N*, *S* and *T*. Both camps routinely report accuracies above 95% on curated alphabet datasets, so the more discriminating practical questions are about *robustness* and *efficiency* rather than headline accuracy, the axes we foreground here. We choose the pixel-based route as it does not require an auxiliary model at inference and reveals the raw representational gap between scratch and transferred features which is the focus of our ablation.

III. METHODOLOGY

A. Problem formulation

We treat fingerspelling recognition as supervised multi-class image classification. Let $\mathbf{x} \in \mathcal{X} = \mathbb{R}^{H \times W \times 3}$ be an RGB image and $y \in \mathcal{Y} = \{1, \dots, C\}$ its label, with $C = 37$. We seek a parametric classifier $f_{\theta} : \mathcal{X} \rightarrow \Delta^{C-1}$ mapping an image to the probability simplex. The network produces a

logit vector $\mathbf{z} = g_{\theta}(\mathbf{x}) \in \mathbb{R}^C$, converted to class probabilities by the softmax

$$\hat{y}_c = \text{softmax}(\mathbf{z})_c = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}, \quad \sum_{c=1}^C \hat{y}_c = 1. \quad (1)$$

Given a training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ drawn i.i.d. from an unknown distribution $\mathcal{D}$, the ideal objective is the expected risk (in class we called it the cost function) $R(\theta) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}}[\ell(f_{\theta}(\mathbf{x}), y)]$, which is intractable. We instead minimise the *empirical risk*

$$\hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}_i), y_i), \quad (2)$$

with a loss ℓ defined in Section III-F. Regularisation, augmentation and transfer learning are all devices for keeping the gap $R - \hat{R}$ small when N is limited.

B. Data analysis

The label space contains 37 classes. The ASL sign for 0 is visually identical to the letter *O* and was excluded (also, the amount of data of 0 were too small, compared to other classes), as were the `del` and `space` folders that do not appear across all sources; the `nothing` label was normalised to *blank*. Five public datasets (Table I) are merged into a single pool *before* splitting, so every source contributes to every split.

TABLE I: Source datasets unified into the benchmark.

Dataset	Type	Native size	Classes
ASL Alphabet [21]	Video frames	200×200	A-Z, blank
Combined ASL [22]	Real, uniform	400×400	A-Z, 1-9
ASL-HG (Mendeley) [23]	Real, in-wild	Variable HD	A-Z, 1-9
Synthetic Alphabet [24]	Synthetic	Variable	A-Z, blank
Synthetic Numbers [25]	Synthetic	Variable	1-10, blank

Near-duplicate frame thinning. Each image is opened and verified to be at least 32×32 px; corrupt files are dropped. A stratified 70/15/15 split is then applied per class with a fixed seed (33), guaranteeing at least one image per class per split. This yields 33 727 training, 7231 validation and 7225 test images (48 183 total). Class counts are uneven: *blank* is over-represented (1422 training images) while the digits have ~ 700 -720 each. To prevent the loss from being dominated by frequent classes, training uses inverse-frequency class weights

$$w_c = \frac{N}{C n_c}, \quad (3)$$

where N is the total training count, $C = 37$ and n_c the count of class c ; rare classes thus receive proportionally larger gradients.

C. Convolutional feature extraction

All models are convolutional networks. A 2-D convolutional layer convolves (technically cross-correlates) an input feature map $\mathbf{X}$ with a bank of learnable kernels $\mathbf{K}$:

$$(\mathbf{X} * \mathbf{K})_{i,j} = \sum_m \sum_n \sum_c \mathbf{X}_{i+m, j+n, c} \mathbf{K}_{m,n,c} + b. \quad (4)$$

Three properties motivate this choice over fully-connected layers: *local receptive fields* (each output depends only on a spatial neighbourhood), *parameter sharing* (the same kernel is applied at every location, drastically reducing parameters), and *translation equivariance* (shifting the input shifts the response), which is appropriate because a hand sign retains its meaning regardless of its position in the frame. Each convolution is followed by a pointwise nonlinearity, the Rectified Linear Unit

$$\sigma(u) = \max(0, u), \quad (5)$$

which is cheap, non-saturating for $u > 0$, and mitigates the vanishing-gradient problem of sigmoid units. Spatial pooling (max- or average-pooling) progressively reduces resolution, enlarges the effective receptive field and grants limited translation *invariance*. Our convolutional blocks follow the order Conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool.

D. Batch Normalisation

Batch Normalisation (BN) [12] normalises each channel over a mini-batch $\mathcal{B}$ before scaling and shifting it with learnable parameters γ, β :

$$\mu_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (x_i - \mu_{\mathcal{B}})^2, \quad (6)$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta. \quad (8)$$

By keeping the distribution of each layer's inputs approximately zero-mean and unit-variance, BN reduces the dependence of each layer on the parameters of all preceding layers (originally framed as reducing internal covariate shift), smooths the loss landscape, and permits higher learning rates. The mini-batch statistics also add a data-dependent noise that acts as a regulariser. At inference, μ and σ^2 are replaced by running estimates accumulated during training, so the mapping becomes deterministic. As Section V shows, BN is the single most impactful ingredient for our scratch networks.

E. Dropout and its interaction with BN

Dropout [13] regularises a layer by multiplying its activations $\mathbf{h}$ by a random binary mask during training. Using the common inverted-dropout convention,

$$\tilde{h}_j = \frac{1}{1-p} m_j h_j, \quad m_j \sim \text{Bernoulli}(1-p), \quad (9)$$

where p is the drop probability and the factor $1/(1-p)$ keeps the expected activation unchanged so that no rescaling is needed at test time. Dropout discourages co-adaptation of features and can be interpreted as implicitly averaging an exponential ensemble of sub-networks. Crucially, BN and Dropout interact unfavourably when stacked: Dropout changes the variance of activations between training and inference, whereas BN's running statistics assume a fixed variance, producing a *variance shift* that degrades the normalisation at test time [12], [13]. This theoretical tension predicts the empirical result in Section V, where adding Dropout on top of BN slightly hurts accuracy.

F. Loss function

We use the class-weighted cross-entropy loss, the negative log-likelihood of the correct class under (1):

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{c=1}^C w_c y_c \log \hat{y}_c = -w_y \log \hat{y}_y, \quad (10)$$

with one-hot $\mathbf{y}$ and weights w_c from (3). Minimising (10) is equivalent to minimising the Kullback-Leibler divergence (called in class KL divergence) between the (weighted) empirical label distribution and the model’s predictive distribution, i.e. to maximum-likelihood estimation under a categorical model.

G. Optimisation

All networks are trained by stochastic gradient descent with the Adam (adaptive moment) optimiser [16], which adapts per-parameter step sizes using exponential moving averages of the gradient g_t and its square:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (11)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (12)$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \quad (13)$$

where $\hat{m}_t, \hat{v}_t$ are bias-corrected estimates and η_t is the learning rate. We add L_2 weight decay (10^{-4}) to the objective. The learning rate follows a cosine-annealing schedule [17]

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{t}{T} \pi\right), \quad (14)$$

decaying smoothly from $\eta_{\max}$ to $\eta_{\min} = 10^{-6}$ over T epochs, which improves late-stage convergence. To prevent exploding gradients we clip the global gradient norm,

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|_2}\right), \quad c = 1, \quad (15)$$

and use automatic mixed precision (FP16/FP32) [18] to reduce memory and wall-clock cost¹. Table II summarises the per-group configuration.

TABLE II: Training configuration per experiment group.

Group	Normalisation	Augment.	LR	Epochs
Scratch (no aug)	Dataset	No	10^{-3}	50
Scratch (+Aug)	Dataset	Yes	10^{-3}	50
Transfer - Phase 1	ImageNet	Yes	10^{-3}	5 (frozen)
Transfer - Phase 2	ImageNet	Yes	10^{-4}	up to 50

¹Automatic Mixed Precision (AMP) reduces memory overhead and training time by using a lower precision for values that does not require high accuracy (i.e. weights, activations...) and keeping high precision for gradients. This theoretically should not affect noticeably the results (metrics). This is similar to the method used by the group previous to our presentation, but we used the integrated method instead, which only works with CUDA (NVIDIA GPUs).

H. Data augmentation as regularisation

Augmentation enlarges the support of the training distribution with label-preserving transforms, a form of vicinal-risk minimisation that reduces over-fitting and encourages invariance to nuisance factors. The complete training transform pipeline is detailed in Table III. Crucially, *vertical flips and 90° rotations are excluded*². Vertical flips are unnatural (signing upside down), and rotation is kept small because J is essentially an inclined variant of I , and large rotations would blur that distinction. The evaluation/inference transform is deterministic (resize and normalise only).

TABLE III: Data Augmentation Pipeline and Hyperparameters

Transform Layer	Hyperparameters / Configuration
T.Resize	Target size: 224×224 pixels
T.RandomAffine	Degrees: $[-15^\circ, 15^\circ]$, Translation: $\pm 10\%$, Shear: $[-10^\circ, 10^\circ]$
T.ColorJitter	Brightness: 0.3, Contrast: 0.3, Saturation: 0.2, Hue: 0.05
T.GaussianBlur	Kernel size: 3, Sigma range: $[0.1, 1.5]$
T.ToImage / T.ToDtype	Cast to <code>torch.float32</code> , Scale intensity to $[0.0, 1.0]$
T.RandomErasing	Prob. (p): 0.2, Scale: $[0.02, 0.10]$, Aspect ratio: $[0.3, 3.3]$
T.Normalize	Scratch Models (Dataset-specific): $\mu = [0.4876, 0.4538, 0.4196]$, $\sigma = [0.2514, 0.2589, 0.2716]$ Transfer Learning (ImageNet): $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$

I. Architectures

All networks output a 37-dimensional logit vector. Two broad architectural families are compared: custom CNNs trained from random initialization (scratch), and pretrained ImageNet backbones adapted via transfer learning.

Scratch CNNs. The scratch family is designed to isolate the effects of network capacity and regularization. The structural flow of these models is consolidated in Fig. 1. A *Baseline* two-block CNN ($3 \rightarrow 32 \rightarrow 64$ channels, global pooling, two-layer classifier; 0.15 M parameters) provides a performance floor. To increase capacity, we extend this to a four-block *Deep CNN* ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ channels; 1.45 M parameters). From this core Deep CNN, four regularized variants are systematically derived: *+Dropout* (rate 0.5 in the classification head), *+BatchNorm* (injected immediately after every convolution, before the ReLU), *+BN+Dropout*, and *+BN+Dropout+Aug*, the last adding the full augmentation pipeline described in Section III-H.

²We also excluded the horizontal flip, although as you explained in class, a left handed person could signed with the other hand, which is equivalent to mirroring the image. This was not taken into account, therefore constituting a flaw in the augmentation pipeline.

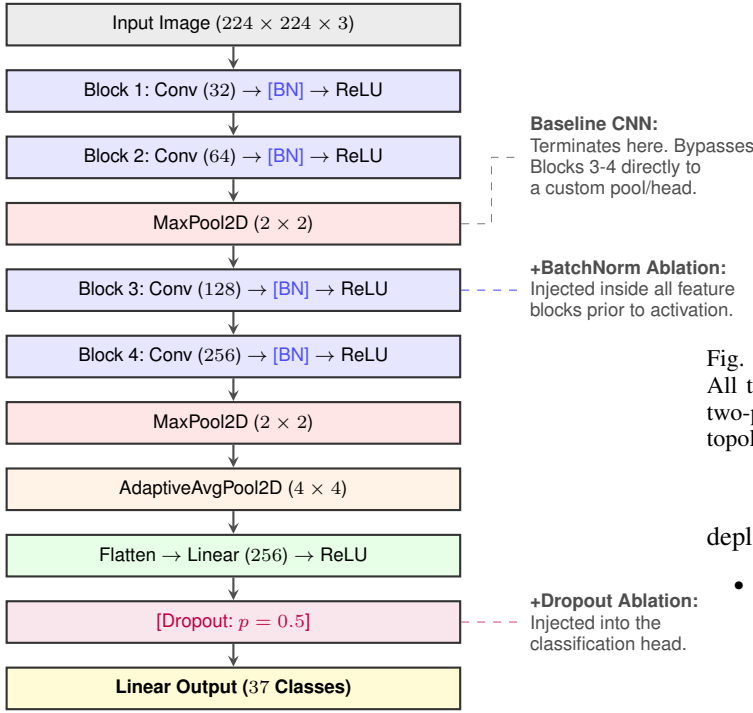


Fig. 1: Consolidated architecture of the Scratch CNN family. The core sequence details the Deep CNN; annotations denote the Baseline structural cut-off and the specific injection points for the Batch Normalization and Dropout regularisation ablations.

Why transfer learning. Training a deep network from scratch requires enough labelled data to estimate millions of parameters without over-fitting, and forces the optimiser to rediscover generic low-level filters (edge, colour and texture detectors) from random initialisation on every new task. In the small-to-medium data regime this is wasteful and unstable. Transfer learning addresses both issues. By initialising from a state θ_0 pretrained on ImageNet, we start inside a good basin of the loss landscape. This reduces the effective number of parameters to estimate, injects a strong inductive bias toward natural-image statistics, and places optimisation in a smoother region, converging much faster than scratch models.

Two-phase fine-tuning. Unfreezing the whole network immediately at a normal learning rate would back-propagate large, noisy gradients from the randomly-initialised head through the pretrained backbone, triggering catastrophic forgetting. As illustrated in Fig. 2, we employ a two-phase strategy. In *Phase 1*, the backbone is strictly frozen ($\nabla_{\theta_0} = 0$) and only the new classification head is trained for 5 epochs at $\eta = 10^{-3}$. Once the head is calibrated, *Phase 2* unfreezes the backbone and fine-tunes the entire pipeline end-to-end at a $10\times$ smaller rate (10^{-4}), specialising the features to ASL without destroying generic low-level filters. The substituted head is deliberately minimal (Dropout $\rightarrow$ Linear) because a rich pretrained representation needs only a linear decision boundary.

Why these four backbones. The chosen backbones span the two axes critical for this study: representational capacity and

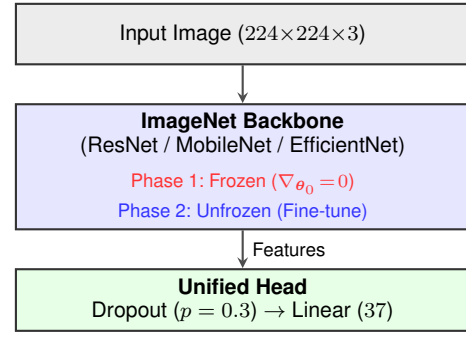


Fig. 2: Conceptual schematic of the Transfer Learning pipeline. All tested backbones share the same unified classification head and two-phase fine-tuning protocol, isolating the effect of the backbone topology.

deployment cost.

- *ResNet-18 and ResNet-50* [9] share a residual design but differ by $2.3\times$ in parameters (11.2 M vs. 25.6 M). ResNet composes residual blocks that learn a mapping added to an identity shortcut:

$$y = \mathcal{F}(x, \{W_i\}) + x, \quad (16)$$

enabling reliable optimisation. ResNet-50 specifically stacks bottleneck blocks to keep computation tractable. Testing both isolates the effect of depth, whether increasing the architecture’s depth improves significantly the performance of the pretrained model.

- *MobileNetV3-Small* [10] (2.5 M) represents the mobile-efficiency extreme (good for deployment). It replaces standard convolutions with depthwise-separable convolutions, reducing cost by:

$$\frac{1}{N} + \frac{1}{D_k^2}, \quad (17)$$

yielding an $8\text{-}9\times$ saving. It tests how much accuracy survives when the parameter budget is cut by $10\times$ relative to ResNet-50.

- *EfficientNet-B0* [11] (5.3 M) uses compound scaling to balance depth, width, and input resolution harmoniously. It tests whether principled scaling beats heuristically-sized ResNets at a moderate hardware budget. It also serves as a baseline within the pretrained models.

All four are (a) pretrained available in torchvision, (b) small enough to fine-tune quickly, and (c) widely used and well-understood baselines, which keeps the comparison reproducible and the conclusions transferable. By selecting one model from each of the dominant design philosophies (plain residual depth, mobile efficiency and balanced compound scaling) we are able to not just answer “does transfer learning win?” but the sharper deployment question “what *kind* of backbone works best per parameter for real-time ASL recognition, and offers the greatest trade-off?”.

J. Evaluation metrics

Because the classes are imbalanced, accuracy alone is misleading. For each class c we compute precision and recall from the true/false positives and negatives,

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad (18)$$

and combine them with their harmonic mean, the F1 score,

$$F1_c = \frac{2P_cR_c}{P_c + R_c}. \quad (19)$$

We summarise performance with the *macro-averaged* F1 and the *balanced accuracy*, both of which weight every class equally regardless of frequency:

$$\text{macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c, \quad \text{bal. acc.} = \frac{1}{C} \sum_{c=1}^C R_c. \quad (20)$$

For deployment we additionally report each model’s parameter count as a hardware-agnostic proxy for its memory footprint and computational cost. We deliberately do not use wall-clock training or inference time as a comparison axis: all training and timing were performed on H100 GPUs of a shared HPC cluster, so such numbers would neither reflect the commodity hardware targeted by the application nor constitute a fair cross-model comparison. Deployment feasibility is instead argued from parameter count and demonstrated directly through the real-time system of Section V-A.

K. Implementation and reproducibility

All models are implemented in PyTorch with torchvision backbones [19]. Every stochastic component is seeded with a single global seed (33): the dataset split, the data-loader shuffling, the parameter initialisation and the augmentation sampler, so that runs are reproducible up to the non-determinism of GPU convolution kernels. Training was performed on the CSUC Pirineus 3 cluster, each job requesting one GPU through a SLURM batch script. The deterministic resize-and-normalise transform used at validation and test time is identical across all models, so reported metrics are directly comparable. Checkpoints are selected by best validation macro-F1, and the selected checkpoint is evaluated once on the test split to avoid test-set leakage through model selection.

Sharding for cluster I/O

Stored naively, the unified pool is a directory tree of roughly 48 000 individual JPEG files. On a shared HPC file system this layout is pathological: every image is a separate `open/read/close` cycle, and the per-file metadata lookups and random disk seeks dominate the actual pixel transfer. With many data-loader workers issuing such requests in parallel across the network file system, the input pipeline becomes I/O-bound and the GPU starves between batches, so wall-clock time is spent waiting on the file system rather than computing gradients. The cost grows with the number of files, not their size, which is exactly the regime a frame-level image dataset falls into.

To avoid this, `prepare_data.py` serialises the pool into *WebDataset* shards: each split is packed into a sequence of `.tar` archives of about 1000 images each (`shard_size` in Table II), with every sample written as a `{key}.jpg` payload paired with a `{key}.cls` label record, and a `per-split_info.json` listing the shard files, the total count and the per-class counts. At training time the shards are streamed *sequentially*: a worker reads one tar end-to-end as a contiguous block, converting hundreds of thousands of small random reads into a handful of large sequential ones, which is the access pattern networked storage is optimised for. Randomness is recovered cheaply without random seeks by shuffling the shard order and drawing samples through an in-memory shuffle buffer, and shards are distributed across data-loader workers so throughput scales with worker count. This keeps the pipeline compute-bound and lets the large cluster batch size (256) be fed without a data-loading bottleneck, while leaving the on-disk split (Section III-B) and the deterministic transforms (Section III-H) byte-for-byte identical to the local CSV-based variant used for development.

IV. EXPERIMENTS

The experiments are designed as a sequence of controlled comparisons rather than a hyper-parameter search:

- 1) **Performance floor and capacity.** Baseline CNN versus Deep CNN isolates the effect of depth/capacity alone.
- 2) **Regularisation ablation.** Starting from the Deep CNN we add Dropout, Batch Normalisation, both, and finally augmentation, to attribute the contribution of each. The non-augmented variants deliberately use the deterministic *evaluation* transform during training, so that raw model capacity and regularisation are not confounded with augmentation.
- 3) **Transfer learning.** The four ImageNet backbones are fine-tuned with the two-phase schedule of Section III-I, all sharing the same head, loss and optimiser.

Every model is evaluated once on the held-out test set using the metrics of Section III-J.

V. RESULTS

Table IV reports the test-set metrics for all ten models, ordered by macro-F1, and Fig. 3 visualises the three headline metrics. The gap between the two families is large and consistent: every transfer model exceeds 98.9% macro-F1, while the best scratch model reaches 95.8% and the baseline floor is 69.3%.

Reading the table. Three observations follow from Table IV. First, for every model the three quality metrics (accuracy, macro-F1, balanced accuracy) agree to within a few thousandths; because the class-weighted loss of (10) prevented the dominant blank class from skewing learning, no model achieves high accuracy by neglecting rare classes, so the simple accuracy and the imbalance-aware metrics coincide. Second, the four transfer models are separated by only 0.7 macro-F1 points (98.9-99.6%), a spread comparable to the run-to-run noise expected from a single split, so their ranking

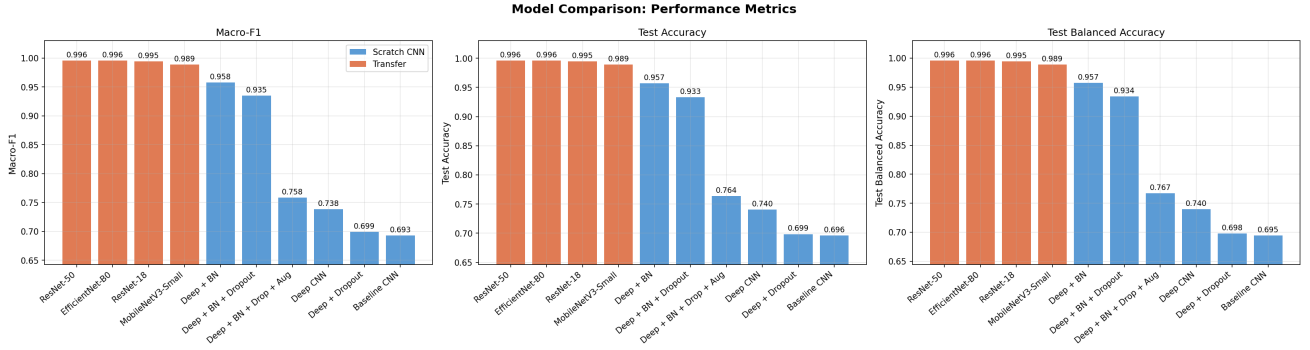


Fig. 3: Headline test-set metrics (accuracy, macro-F1, balanced accuracy) for all ten models. Transfer models (orange) cluster near saturation; among scratch models (blue), Batch Normalisation is the decisive ingredient.

TABLE IV: Test-set results, ordered by macro-F1. “Epochs” for transfer models denotes Phase 1+Phase 2 (frozen-backbone adaptation followed by end-to-end fine-tuning).

Model	Type	Params (M)	Accuracy	Macro-F1	Balanced Acc.	Epochs
ResNet-50	Transfer	25.56	0.9961	0.9959	0.9959	5+25
EfficientNet-B0	Transfer	5.29	0.9960	0.9957	0.9957	5+47
ResNet-18	Transfer	11.18	0.9949	0.9946	0.9945	5+38
MobileNetV3-Small	Transfer	2.54	0.9895	0.9892	0.9892	5+50
Deep + BN	Scratch	1.45	0.9571	0.9576	0.9575	50
Deep + BN + Dropout	Scratch	1.45	0.9333	0.9349	0.9343	50
Deep + BN + Dropout + Aug	Scratch	1.45	0.7640	0.7585	0.7671	50
Deep CNN	Scratch	1.45	0.7405	0.7384	0.7400	50
Deep + Dropout	Scratch	1.45	0.6985	0.6992	0.6980	50
Baseline CNN	Scratch	0.15	0.6965	0.6929	0.6946	50

should be read as indicative rather than significant, and model choice among them is better decided on efficiency. Third, within the scratch family the ordering is highly informative: the +0.22 jump from Deep CNN to Deep CNN + BN dwarfs every other ablation step, while Dropout moves the score by less than its likely noise. The rare digit 10 (only 150 test images) is recognised essentially perfectly by all transfer models, confirming that the inverse-frequency weighting and the stratified split together kept even the smallest class learnable.

Training dynamics. Fig. 5 overlays the learning curves of all ten models and reveals four qualitatively distinct phases. (i) The transfer models (magenta/blue) show the characteristic two-phase signature: a flat head-only Phase 1, then a near-vertical jump at the phase boundary (dotted line) the moment the backbone is unfrozen, then saturation, the sign of *adapting* pretrained features rather than learning them from scratch. (ii) The best scratch model, Deep CNN + BN, climbs smoothly and monotonically to ~ 0.96 with train and validation almost aligned, indicating healthy optimisation with little over-fitting. (iii) The augmented variant (dashed teal) is the lowest curve, still rising at the epoch budget and with validation sitting above training, the fingerprint of an under-trained, heavily regularised model rather than an over-fit one. (iv) The shallow Baseline CNN plateaus early below 0.70, the expected ceiling of a low-capacity network. These four shapes (adapt, converge, under-train, saturate) recur across the remaining models and frame

the discussion in Section VI.

Per-class behaviour. Fig. 6 reports per-class F1 for every model. Transfer models are near-perfect across all 37 classes, including the rare digit “10”. The weak scratch models struggle most on visually similar, closed-fist signs (e.g. *M*, *N*, *S*, *T*) and on letters such as *D* and *E*, while blank is consistently the easiest class (most distinct and most data). The augmented variant exposes the under-training most sharply, with isolated failures (e.g. *M* at F1= 0.23, digit 9 at 0.50, *D* at 0.54) that the same architecture resolves once Batch Normalisation alone is used.

A closer look at the hard classes. Table V isolates the F1 of nine representative classes for four models spanning the quality range. The pattern is uniform and striking. The plain Deep CNN scores in the 0.6-0.8 band on the closed-fist group and on the rotation-sensitive *D/E* pair; adding Batch Normalisation lifts every one of these into the 0.93-0.96 band, a +0.2-0.3 swing concentrated exactly on the classes that were hard. The augmented column is highly non-uniform: it collapses on *M* (0.23) and digit 9 (0.50) yet stays competitive elsewhere, which is the per-class fingerprint of *under-training* rather than a uniform loss of capacity. ResNet-18 sits at 0.99-1.00 on every row, including the rare digit 10 and the otherwise-difficult closed-fist signs, showing that the transferred features separate precisely the configurations the scratch models cannot.

TABLE V: Per-class test F1 for nine representative classes across four models. BN rescues the hard classes; the augmented model fails non-uniformly (a signature of under-training); the transfer model is near-perfect.

Class	Deep CNN	Deep+BN	+Dr+Aug	ResNet-18
<i>M</i>	0.64	0.94	0.23	0.99
<i>N</i>	0.69	0.95	0.53	0.99
<i>S</i>	0.65	0.95	0.69	0.99
<i>T</i>	0.68	0.94	0.74	0.99
<i>D</i>	0.71	0.93	0.54	0.99
<i>E</i>	0.65	0.95	0.62	1.00
digit 9	0.78	0.96	0.50	1.00
digit 10	0.61	0.99	0.92	1.00
<i>blank</i>	0.88	1.00	0.95	1.00

Where the errors go. Per-class F1 tells us *which* classes are hard but not *how* they are confused. The row-normalised confusion matrices in Fig. 7 make the error structure explicit. For the weak scratch models the off-diagonal mass is not random: it concentrates in compact blocks of visually adjacent signs, most prominently the closed-fist family (*M*, *N*, *S*, *T*), whose only distinguishing cue is thumb placement, and orientation-similar pairs such as *D/E*. The augmented, under-trained model exhibits the sharpest single off-diagonal entry, systematically predicting *N* when shown *M*. In stark contrast, the four transfer models are visually diagonal: their residual errors are isolated single pixels rather than structured blocks, consistent with the near-saturated per-class F1 of Fig. 6. This supports the view that the scratch models fail by *confusing fine-grained hand configurations*, a discriminative-feature problem that ImageNet pre-training largely solves.

Accuracy/size trade-off. Fig. 4 plots macro-F1 against model size. Accuracy saturates quickly with capacity: the four transfer backbones are statistically indistinguishable (98.9-99.6%) despite a $10\times$ range in parameters. ResNet-18 lands within 0.13 F1-points of the much larger ResNet-50 at under half its parameters, while MobileNetV3-Small delivers 98.9% with only 2.54 M parameters, the smallest footprint of any transfer model. Among scratch models the Deep CNN+BN reaches 95.8% at just 1.45 M parameters, a strong result for a network trained from random initialisation.

A. Real-time deployment demonstration

To confirm that the accuracy reported above translates into a usable system, and to substantiate the deployment claim without relying on cluster timings, we built a live recognition demonstrator around MobileNetV3-Small, the smallest transfer model. The pipeline reads frames from a standard laptop webcam through OpenCV (`cv2.VideoCapture`), crops a fixed central region-of-interest box that guides the signer, applies the exact deterministic evaluation transform of Section III-H (resize and normalise, no augmentation), and runs a single forward pass per frame. The top-three predictions are overlaid with their softmax probabilities, colour-coded by the output of the softmax layer (green $\geq 70\%$, yellow $\geq 40\%$, red $< 40\%$) so the user can immediately read how certain the model is. Running on commodity CPU hardware, the system

sustains an interactive frame rate suitable for fingerspelling, demonstrating that the 2.54 M-parameter backbone is light enough for real-world, sensor-free deployment on the kind of hardware an end user actually owns.

VI. DISCUSSION

Transfer learning dominates, but read 99% with caution.

The ~ 30 -point gap between transfer and scratch models confirms that ImageNet features transfer extremely well to hand images, and that fine-tuning on a few thousand images per class is enough to specialise them. However, near-perfect numbers should be interpreted in light of how “easy” this particular benchmark is: two of the five sources are synthetic with clean backgrounds, the real datasets have largely uniform backgrounds, frames originate from a limited number of signers, and a small amount of near-duplicate leakage remains. The test set is therefore *in-distribution*, so $\hat{R}$ in (2) underestimates the true risk R on unseen signers, cluttered backgrounds and different cameras. The ranking across models is informative, but the absolute 99% is a property of the dataset as much as of the models.

Batch Normalisation is the single most impactful choice for scratch CNNs.

Adding BN to the Deep CNN raises macro-F1 from 0.738 to 0.958, by far the largest jump among the scratch ablations, consistent with the optimisation and regularisation benefits of (6) and (8). In contrast, Dropout *alone* provides essentially no benefit (0.699 vs. 0.738) and, layered on top of BN, slightly *hurts* (0.935 vs. 0.958). This is exactly the variance-shift interaction predicted in Section III-E: BN already regularises through mini-batch statistics, so additional head Dropout is partly redundant and disturbs the normalisation statistics at test time.

The augmented model is under-trained, not broken. At first glance the augmented variant (0.759) looks like a failure of data augmentation. The learning curves tell a different story: validation accuracy sits *above* training accuracy (because the model never sees the clean validation distribution during training, only the harder augmented one) and both curves are still climbing at the epoch budget. Strong augmentation combined with Dropout substantially slows convergence, so the reported number is an under-trained *snapshot* rather than the model’s ceiling. With a longer schedule this configuration would be expected to match or exceed the non-augmented BN model, and would likely generalise better out of distribution, precisely the regime the other numbers do not test.

Which model to deploy. Since accuracy is saturated across the transfer models, the deciding factor for deployment is model size rather than a fraction of an F1-point. MobileNetV3-Small is the natural choice: its small footprint, which follows directly from the depthwise-separable saving of (17), costs only 0.7 macro-F1 points relative to ResNet-50 while using roughly a tenth of its parameters, and we verified that it sustains interactive frame rates from a standard laptop webcam in the live system of Section V-A. ResNet-18 is the better pick when the extra accuracy is worth a larger model and the hardware can accommodate it, effectively tying the best

Tradeoff Analysis

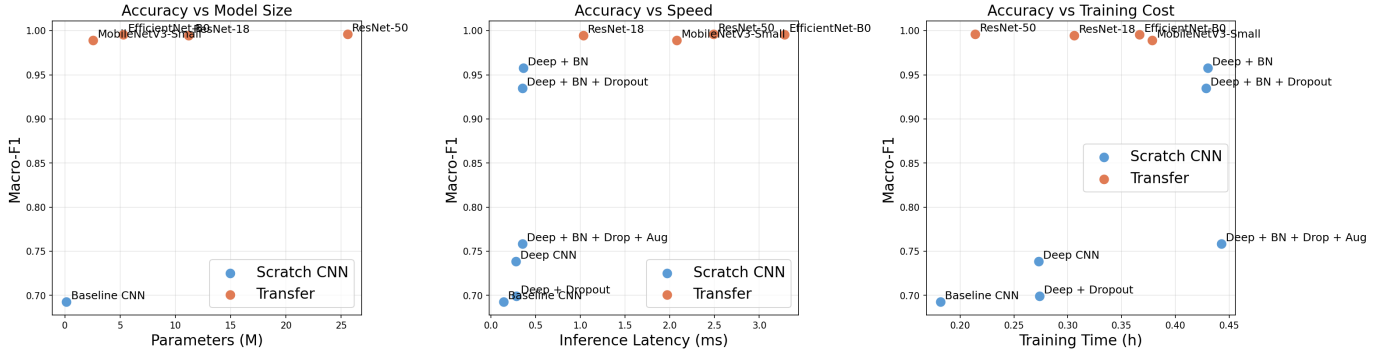


Fig. 4: Test macro-F1 versus parameter count (log scale). Transfer models (orange) form a saturated cluster at the top across a 10× range in size; the scratch Deep CNN + BN (blue) is the strongest small model at 1.45 M parameters.

while using under half of ResNet-50’s parameters. If pretrained weights are unavailable, the scratch Deep CNN + BN remains a credible lightweight fallback at 95.8% macro-F1 with only 1.45 M parameters.

Class weighting works, but cannot manufacture diversity.

The inverse-frequency weighting of (3) successfully prevents the over-represented *blank* class from dominating the loss: balanced accuracy and macro-F1 track plain accuracy closely for every model, so no class is being silently traded away. Weighting cannot, however, create data diversity it does not have: the rare digit classes are learned well here only because, within this clean and partly synthetic pool, they too are visually easy. A genuinely harder benchmark (Section VI-A) would likely re-open the gap between accuracy and macro-F1 that the current data hides.

An error taxonomy that pre-training resolves.

Reading Figs. 6 and 7 together yields a consistent picture. The scratch models’ errors are *structured and semantic*: confusions cluster among hand shapes that differ only in a thumb position or a small rotation (*M/N/S/T*, *D/E*), exactly the fine-grained distinctions a shallow feature extractor struggles to encode. The transfer models inherit ImageNet’s rich mid-level features and reduce these same confusions to near zero, which is why their improvement is largest precisely on the hardest classes rather than uniform across the label space. This is the most informative finding of the study: the benefit of transfer learning here is not a generic accuracy bump but the resolution of a specific, interpretable failure mode.

A. Limitations

Because the headline scores are near-perfect, it is essential to be explicit about the factors that inflate them and bound their interpretation. We group the limitations into data, evaluation-protocol, modelling and measurement concerns.

Data limitations.

- *Synthetic and clean imagery.* Two of the five sources are computer-rendered and the real sources have largely uniform backgrounds and good lighting. The benchmark is therefore substantially “easier” than in-the-wild capture,

and accuracy on cluttered scenes, poor lighting or low-resolution webcams would be markedly lower.

- *Limited signer diversity.* Real frames (non-synthetic) originate from a small number of signers and hand appearances. Skin tone, hand size, jewellery and handedness are under-represented relative to a real user population, so the model may be biased toward the signers it saw.
- *Absolute scarcity of rare classes.* Inverse-frequency weighting ((3)) balances the *loss*, but cannot create diversity: classes such as digit 10 have only ~ 150 test images, so their per-class metrics carry wide confidence intervals.
- *Label-space simplifications.* The sign for 0 (visually identical to *O*) was removed and *del/space* were dropped, so the label space is a clean 37-class subset rather than the full operational vocabulary of a fingerspelling system.

Evaluation-protocol limitations.

- *Single split, no cross-validation.* All numbers come from one stratified 70/15/15 split with a fixed seed. Without repeated splits or multiple seeds we cannot report variance, so small differences, in particular the 0.7-point spread among the transfer models, are within plausible noise and should not be over-interpreted.
- *Not signer-independent.* The split is stratified by class, not by signer, so frames of the same signer (and often the same recording session) appear in both training and test. This is the single largest source of optimism: a signer-independent (grouped) split would almost certainly lower the transfer scores.
- *Residual near-duplicate leakage.* Frame thinning reduces but does not eliminate near-duplicate video frames across the split; the accepted residual ($< 1\%$) pushes every figure slightly upward.
- *In-distribution testing only.* Training and test images are drawn from the same pool, so the empirical risk $\hat{R}$ of (2) estimates in-distribution performance only and is an optimistic proxy for the true deployment risk R .

Modelling limitations.

- *Static, single-frame task.* The motion-dependent letters *J* and *Z* cannot be fully disambiguated from one frame;

they were only partially accommodated by limiting rotation augmentation, and a temporal model would be required to handle them properly.

- *Under-trained augmented configuration.* The augmented model was stopped at the same 50-epoch budget as the others despite still improving, so its reported 0.759 is a snapshot, not a fair estimate of its ceiling; the augmentation ablation is therefore inconclusive rather than negative.
- *Limited augmentation realism.* The augmentation pipeline perturbs geometry and colour but does not synthesise realistic backgrounds, occlusions or motion blur, so it cannot close the gap to in-the-wild data on its own.
- *No confidence calibration.* We report only point predictions. A deployed assistive system must know *when it is unsure* so it can defer or request a repeat; a near-perfect but over-confident classifier can be more harmful in practice than a slightly less accurate but well-calibrated one.

Measurement and reproducibility limitations.

- *No rigorous efficiency benchmark.* Training and timing ran on H100 GPUs, so we report no wall-clock training or inference numbers: these would be both mismatched to the commodity hardware the system targets and unfair across models of different sizes. Deployment efficiency is therefore argued from parameter count and a qualitative real-time demonstration (Section V-A) rather than measured CPU latency or throughput; a controlled on-device latency and energy study remains future work.
- *Single hardware/seed.* Training used one cluster configuration and one seed; GPU kernel non-determinism and hardware differences are not characterised.

VII. CONCLUSIONS

We built a reproducible pipeline for 37-class ASL fingerspelling recognition on a unified five-source image pool and systematically compared scratch CNNs against ImageNet transfer learning within a common theoretical framework. The main findings are: (i) transfer learning clearly outperforms training from scratch (98.9-99.6% vs. $\leq 95.8\%$ macro-F1); (ii) among scratch models, Batch Normalisation is the decisive ingredient, while Dropout alone is ineffective and redundant on top of BN, an outcome anticipated by the BN/Dropout variance-shift analysis; (iii) accuracy saturates rapidly with capacity, so the larger ResNet-50 adds little over ResNet-18, and MobileNetV3-Small is the best lightweight option for deployment, confirmed by a live webcam demonstrator running on commodity hardware; and (iv) the apparently poor augmented configuration is in fact under-trained rather than fundamentally worse.

Methodologically, the study is also a reminder that headline accuracy is only as trustworthy as the benchmark beneath it: the gap we measured between scratch and transfer models is real and interpretable, but the absolute 99% is partly a property of a clean, partly synthetic, in-distribution test set. The most valuable next step is therefore not a larger model but a harder,

signer-independent and in-the-wild evaluation that would let these same metrics speak to deployment.

The principal challenges were unifying heterogeneous sources, controlling near-duplicate frame leakage, and designing an orientation-aware augmentation policy. Promising extensions include a signer-independent evaluation protocol with grouped cross-validation, training and testing on in-the-wild data with cluttered backgrounds, a longer schedule for the augmented configuration to test its conjectured higher ceiling, temporal models (e.g. 3D-CNNs or recurrent heads) for the motion-dependent signs, a landmark-based pipeline such as MediaPipe as a complementary or hybrid approach, confidence calibration for safe deferral, and post-training quantisation/distillation to push the efficient models onto edge devices.

REFERENCES

- [1] World Health Organization, “Deafness and hearing loss,” WHO Fact Sheet, 2024. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>
- [2] Rhode Island Commission on the Deaf and Hard of Hearing, “American Sign Language,” State of Rhode Island, Sep. 8, 2023. [Online]. Available: <https://cdhhr.ri.gov/information-referral/american-sign-language.php>. [Accessed: May 29, 2026].
- [3] R. Rastgoo, K. Kiani, and S. Escalera, “Sign language recognition: A deep survey,” *Expert Syst. Appl.*, vol. 164, p. 113794, 2021.
- [4] L. Pigou, S. Dieleman, P.-J. Kindermans, and B. Schrauwen, “Sign language recognition using convolutional neural networks,” in *Proc. ECCV Workshops*, 2014, pp. 572-578.
- [5] N. Pugeault and R. Bowden, “Spelling it out: Real-time ASL finger-spelling recognition,” in *Proc. IEEE ICCV Workshops*, 2011, pp. 1114-1119.
- [6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998.
- [7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proc. NeurIPS*, vol. 25, 2012, pp. 1097-1105.
- [8] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. ICLR*, 2015.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770-778.
- [10] A. Howard *et al.*, “Searching for MobileNetV3,” in *Proc. IEEE ICCV*, 2019, pp. 1314-1324.
- [11] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019, pp. 6105-6114.
- [12] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. ICML*, 2015, pp. 448-456.
- [13] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929-1958, 2014.
- [14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE CVPR*, 2009, pp. 248-255.
- [15] O. Russakovsky *et al.*, “ImageNet large scale visual recognition challenge,” *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211-252, 2015.
- [16] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [17] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *Proc. ICLR*, 2017.
- [18] P. Micikevicius *et al.*, “Mixed precision training,” in *Proc. ICLR*, 2018.
- [19] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. NeurIPS*, vol. 32, 2019, pp. 8024-8035.
- [20] F. Zhang *et al.*, “MediaPipe Hands: On-device real-time hand tracking,” *arXiv:2006.10214*, 2020.
- [21] Akash, “ASL Alphabet,” Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>

- [22] A. Vignon, "Combined ASL Datasets," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/vignonantoine/combinedasldatasets>
- [23] "ASL-HG: American Sign Language Hand Gesture Image Dataset," Mendeley Data, V1. doi: 10.17632/j4y5w2c8w9.1.
- [24] Lexset, "Synthetic ASL Alphabet," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/lexset/synthetic-asl-alphabet>
- [25] Lexset, "Synthetic ASL Numbers," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/lexset/synthetic-asl-numbers>

VIII. ANNEX

This annex collects, at full size, the larger figures referenced throughout the text (Figs. 5 to 7); the headline-metric bar chart (Fig. 3) and the accuracy/size trade-off (Fig. 4) appear inline in Section V.

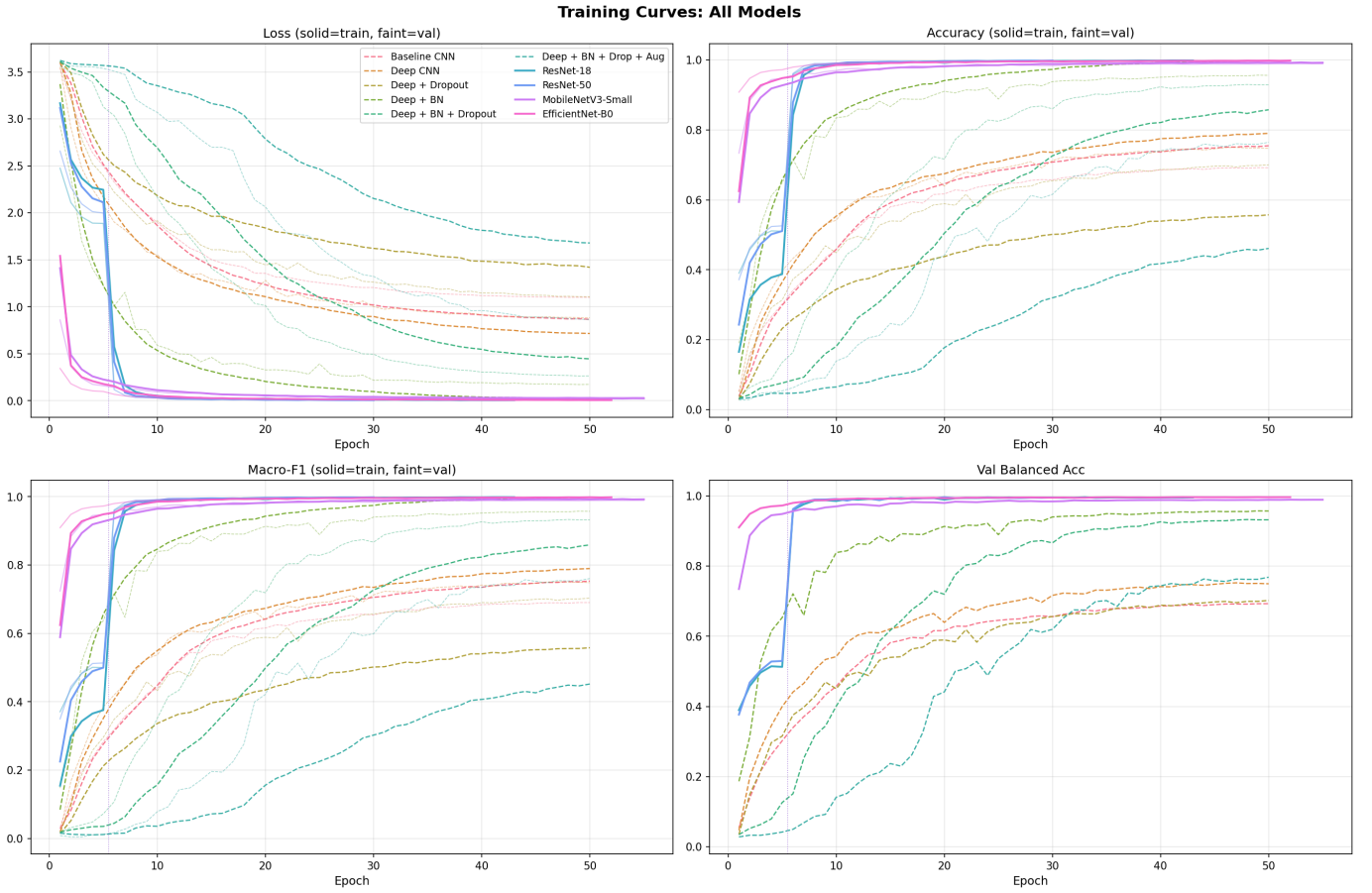


Fig. 5: Training curves for all models (solid = train, faint = validation); the dotted vertical line marks the Phase 1→Phase 2 boundary for transfer models. Transfer backbones jump sharply once unfrozen; the augmented scratch variant (dashed) is still improving at epoch 50.

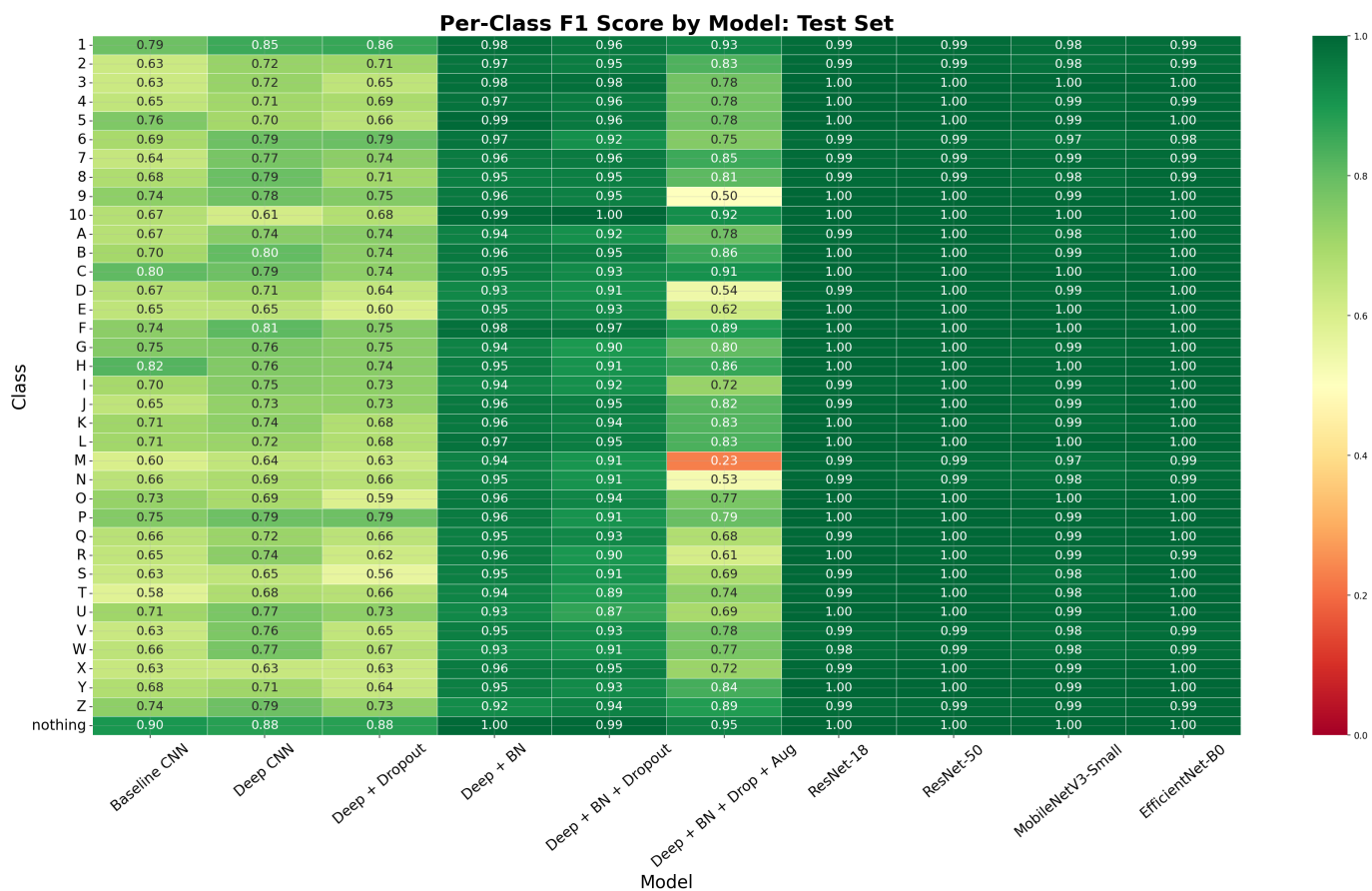
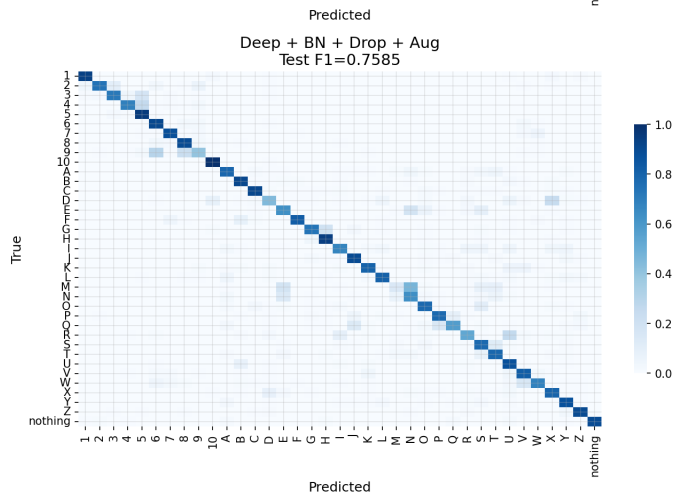
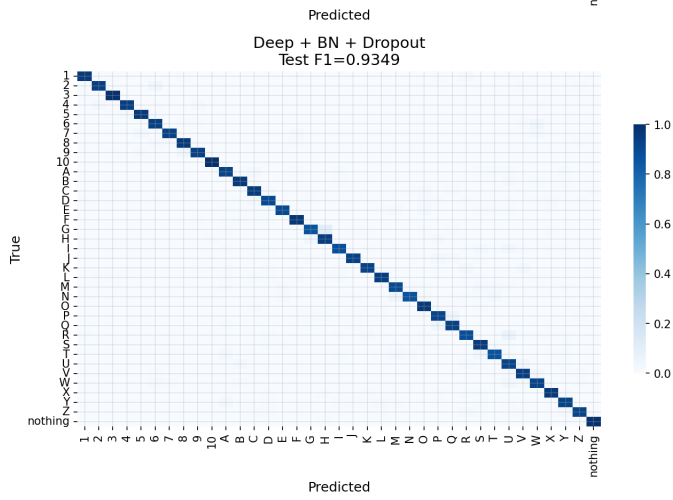
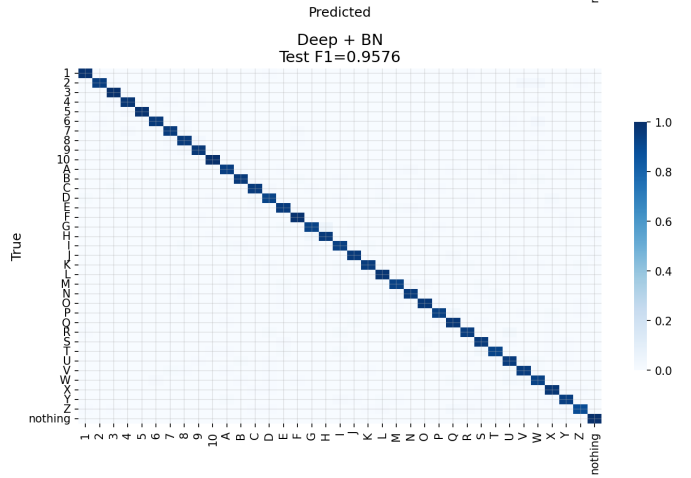
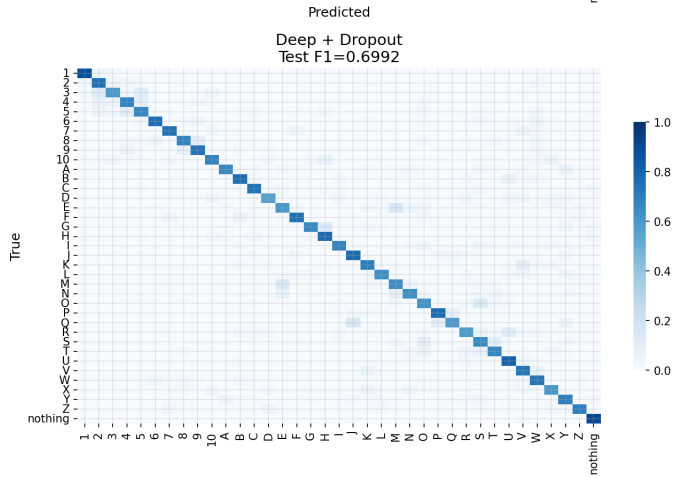
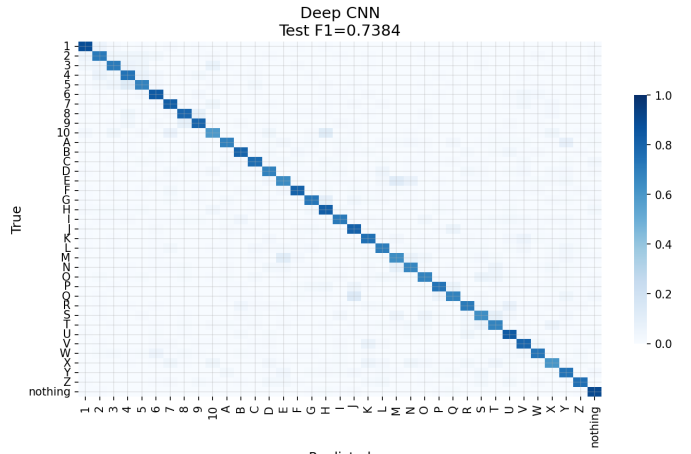
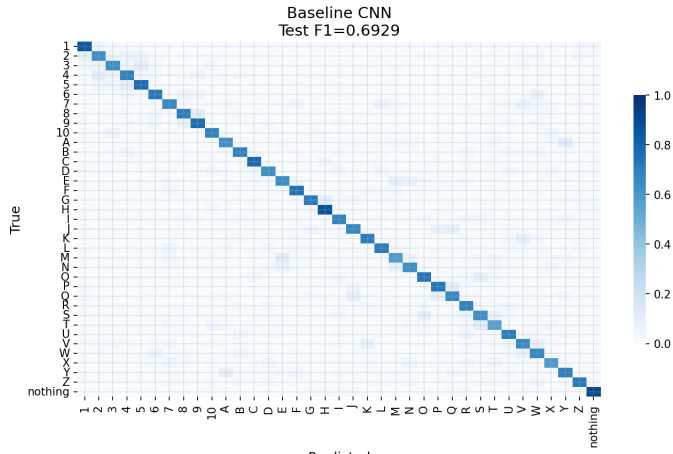


Fig. 6: Per-class test F1 for all ten models (rows shown in the analysis notebook’s native order; columns ordered worst→best). *Note on labels:* this figure uses the notebook’s original tick labels, where the digit rows “0”-“9” correspond to the project’s true digit classes 1-10 and “nothing” denotes the *blank* class.

Confusion Matrices: Test Set (row-normalised)



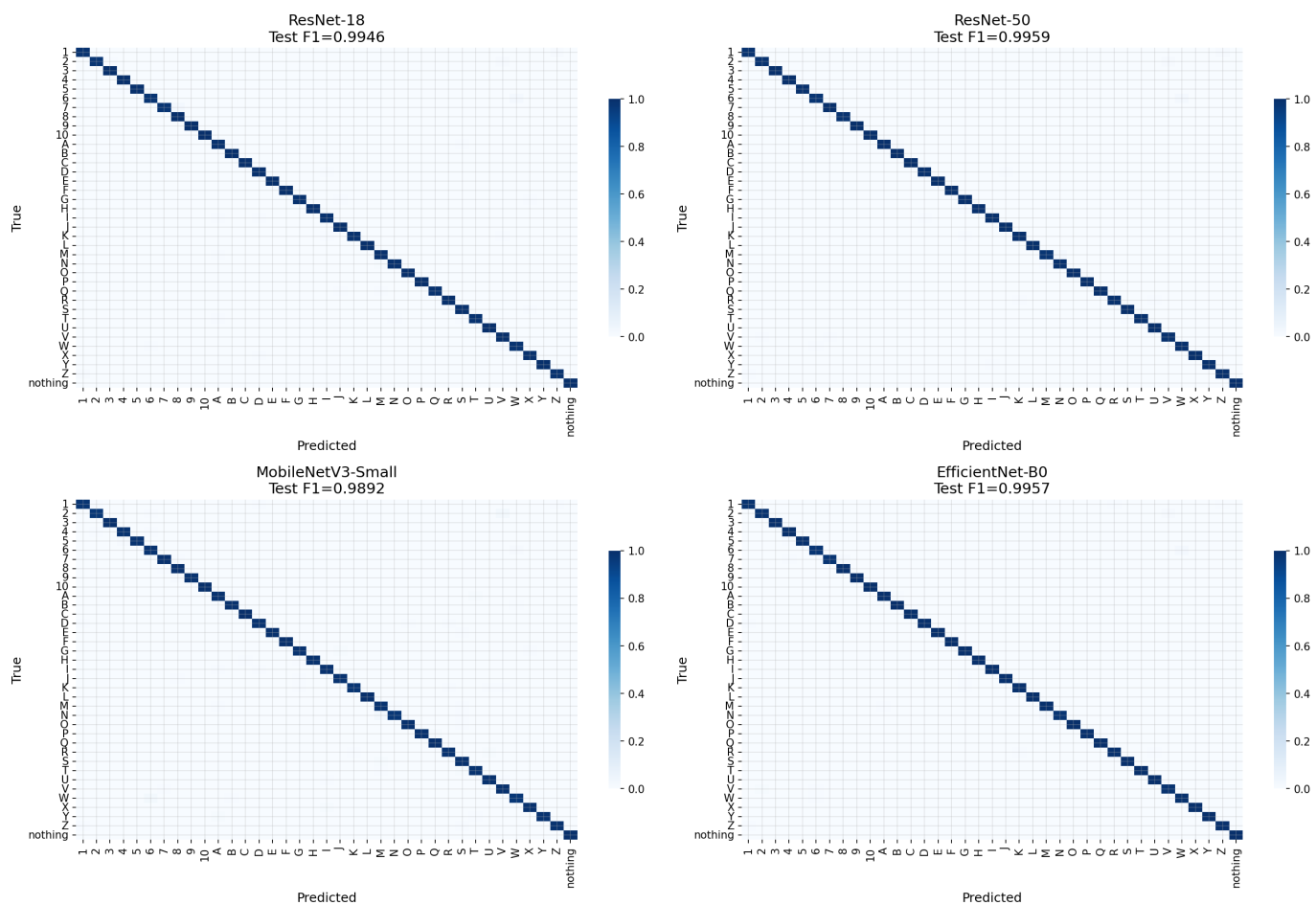


Fig. 7: Row-normalised test-set confusion matrices, one per model (notebook order). Weak scratch models (top rows) show structured off-diagonal blocks among visually similar signs; transfer models (bottom) are essentially diagonal.